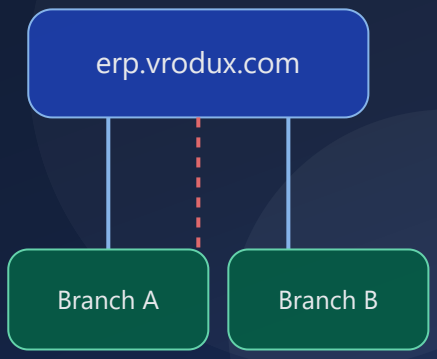
DESIGN PROPOSAL · NOT YET BUILT

Branch Server
Branches that never stop trading

A design for a multi-branch restaurant chain that reports to erp.vrodux.com and keeps every branch fully working when the internet goes down.



Vrodux ERP
Restaurant · POS

Scope
Restaurant + POS branches

Estimate
15–21 weeks

1 The goal in one page

Every branch talks to erp.vrodux.com. When a branch's internet fails, the branch keeps operating, and catches up with head office afterwards.

The idea

Each branch runs a small **branch server** — the same Vrodux server software we already ship for on-premises customers — on a PC in the branch. Every till, tablet, kitchen screen and printer in that branch uses it over the local network. It syncs with the cloud in both directions, but **every kind of data has exactly one owner**, so there is nothing to merge.

Branch owns → flows UP

- Orders, items, payments
- Shifts and cash drawers
- Kitchen tickets, voids, refunds
- Stock movements for its warehouse
- FBR invoice status

Head office owns → flows DOWN

- Menu, prices, modifiers
- Tax settings, branches, tables
- Users, roles, permissions
- Loyalty rules, payment methods
- The licence

Three ways to run a branch

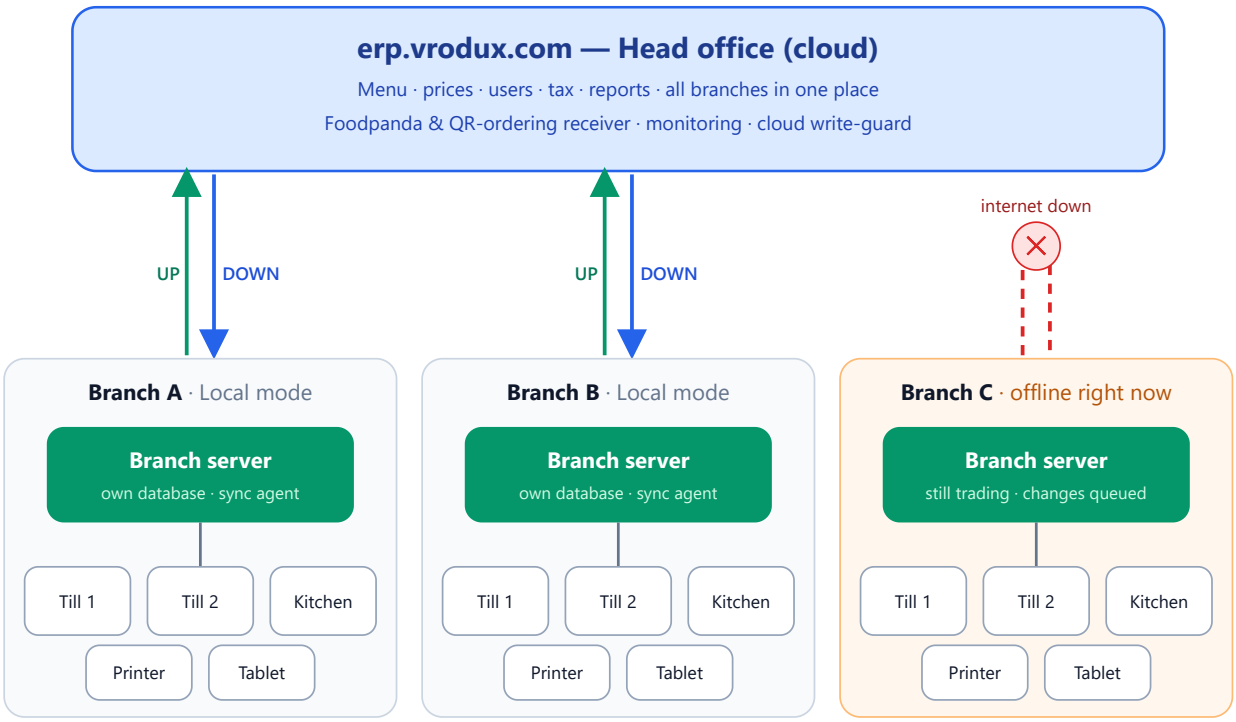
	Today: per-till offline	Standard: branch hub	Recommended: branch server
How it works	Each till queues its own actions in the browser	One local server per branch, synced to the cloud	Same idea, using our own server software
Several tills share open bills offline	**No**	**Yes**	**Yes**
Kitchen screen works offline	**No (prints ticket from till)**	**Yes**	**Yes**
Needs hardware in the branch	**None**	**PC / hub**	**PC + UPS**
Status	**Built**	Industry norm	**To build**
Effort	—	—	15–21 weeks

Cheaper first step (2–3 weeks)

Keep today's per-till offline mode, add head-office monitoring of "last synced" per till, and give each branch a 4G backup link and a UPS. Enough for branches with one till.

2 How it is put together

The branch initiates every connection to the cloud, so no inbound firewall rules and no public address are needed at the branch.



Branches A and B are connected; Branch C lost its internet and carries on locally. All devices in a branch use only that branch's server.

Branch modes

Local Devices use the branch server. The cloud holds a synced copy and refuses direct operational writes for that branch.

Cloud Devices talk to erp.vrodux.com, as today, with the per-till offline queue as a fallback.

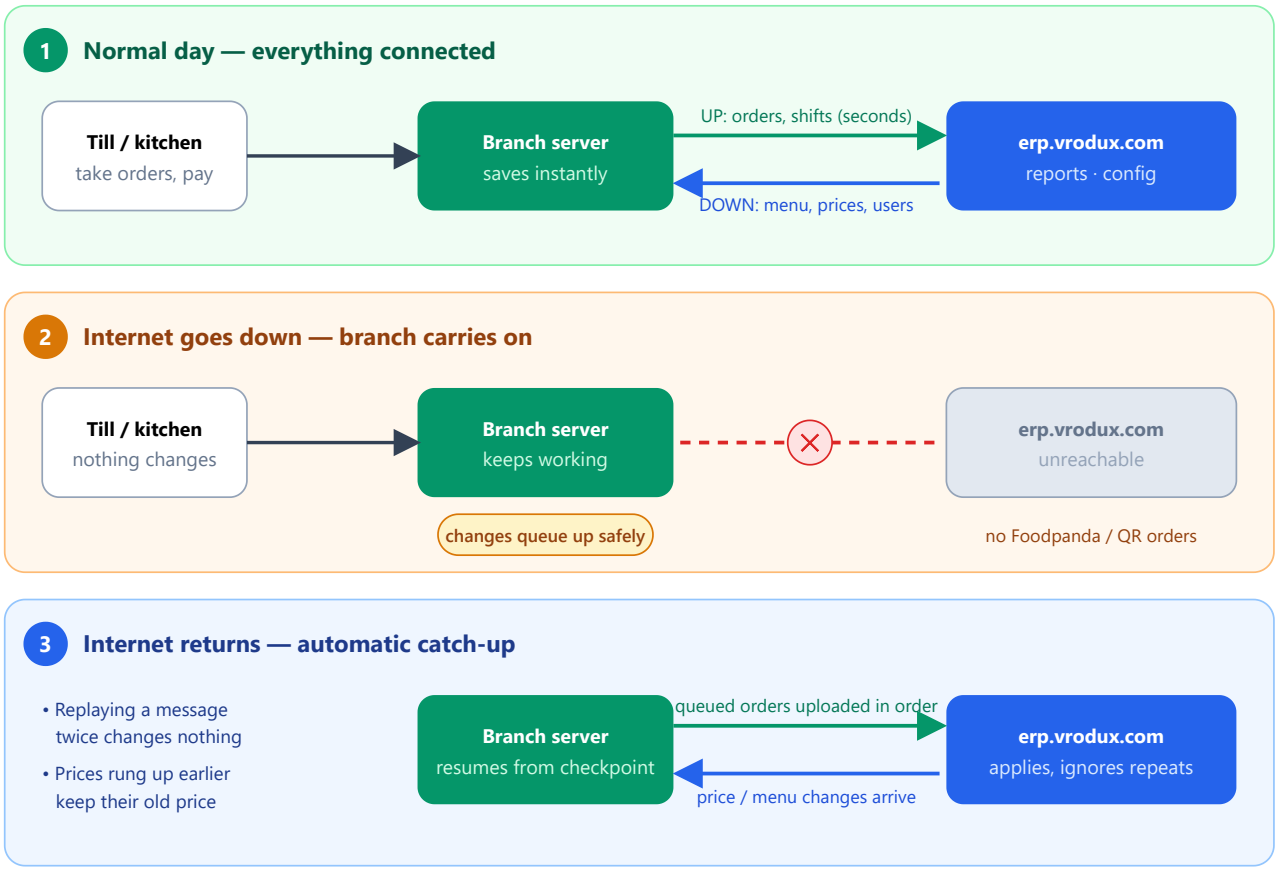
A chain can mix modes, so branches move over one at a time.

Why one writer matters

If both head office and a branch could edit the same bill, every shared record becomes a conflict to resolve. By making the cloud refuse operational edits for a Local branch, the branch stays the only writer. Head office can still read everything and send **instructions** (for example "void this order") that the branch applies itself.

3 What happens during an outage

Staff see no difference. Only the catch-up with head office is delayed.



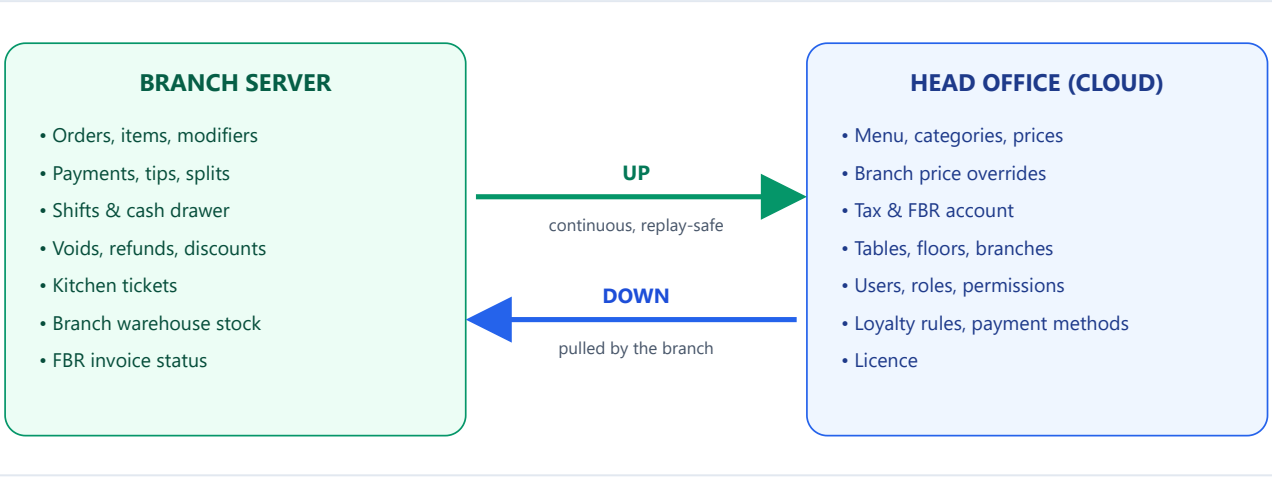
The three states of a branch. The sign-in, tills and kitchen screens depend only on the branch server.

What is refused while offline

Wallet, house-account and loyalty payments (they need the live shared balance). Foodpanda and QR-ordering orders cannot arrive, because the internet cannot reach the branch. FBR invoices are queued and sent by the branch server as soon as the link returns.

4 Who owns what

Because each row has one owner, no merging is needed. Only a few shared items need an explicit rule.



The awkward ones — proposed rules

Data	Why it is awkward	Proposed rule
Customers	Created at any branch, edited at head office	Cloud-owned. A branch creates a *pending* customer; the cloud confirms and sends the final record down
Wallet & loyalty balances	One balance, spent at any branch	Cloud-owned, **online only**. Refused at an offline branch
House-account credit	Shared limit and balance	Same as wallet
Shared stock	Branch stock vs central purchasing	Branch warehouse stock is branch-owned; purchases and transfers arrive down as documents
Table layout vs table status	Edited at head office, used live in the branch	Layout flows down; live status belongs to the branch

Numbers that must never clash

- **Row ids** are GUIDs, so branches can never create the same id.
- **Order and receipt numbers** get a branch-code prefix and a per-day counter held on the branch server, for example `HB2-20261024-0042`. No coordination needed.
- Each branch has its own **time zone and business date**; sync uses UTC instants.

5 How the sync works

Built on what already exists: the cloud-mirror push service, change tracking and the replay-safe client ids.

UP — branch to cloud

- SQL Server change tracking on the in-scope tables
- Runs every few seconds while connected, in bounded batches
- Cloud **upserts by primary key** in dependency order
- Per-branch, per-table checkpoint, so a retry resumes
- These rows are read-only in the cloud and written only by the sync endpoint

DOWN — cloud to branch

- Change feed on config tables, filtered to the branch
- The branch **pulls**; it never needs an inbound connection
- Applied in order, one transaction per batch
- Config is read-only at the branch
- Deletes are already soft deletes, so they flow as ordinary changes

Signing in with no internet

- Users, password hashes, roles, permission grants and denies, and branch assignments flow **down**; a password change at head office reaches the branch within seconds when online.
- The branch server issues its **own sign-in tokens** signed with a branch key, carrying the same claims the cloud uses. Tokens are never exchanged; only data is.
- User administration is done in the cloud, not at the branch.
- A user disabled at head office can still sign in at an offline branch until it reconnects. This can be tightened with a per-branch "last seen" limit.

Traffic that can only reach the cloud

Traffic	How the branch gets it
Foodpanda orders & cancellations	The cloud receives the webhook and queues it; the branch pulls it. Accept, reject and handover go back through the cloud. Offline: no new orders — the branch should show as unavailable on the platform
QR table ordering	Same pull model. The QR page must show a clear "restaurant is offline" state
FBR reporting	The **branch server** reports directly and already queues and retries. An outage only delays it
Online payment callbacks	Cloud-only. Offline branches take cash and terminal card only

Conflict policy

No row conflicts exist because of ownership. For the shared items above the cloud wins and branch writes are proposals. Counters (such as sign-in attempts) use last-writer-wins. Anything found writing on the wrong side is **rejected and logged**, never merged.

6 When things go wrong

The branch server becomes a single point of failure for its branch. The design and the hardware plan have to account for that.

Failure	What happens
Branch internet down	Branch trades normally; uploads and config updates wait. No Foodpanda or QR orders. Wallet, house-account and loyalty payments are refused
Link drops mid-upload	Replayed from the checkpoint; no duplicates
Branch server PC fails	Branch is down. Mitigate with a spare PC with a recent image, an SSD, and a UPS; or switch the branch back to Cloud mode and use per-till offline
Wrong clock on the branch PC	Drift is flagged; sync is refused beyond a limit
Price changed at head office while branch is offline	Applied on reconnect. Bills already rung up keep their price
Two branches, same visible number	Prevented by the branch-code prefix
Cloud outage	Branches are unaffected; they accumulate and catch up
Licence expires	Same offline-licence rules as other on-premises installs

Running it day to day

Install

The existing server installer in "branch mode" asks for the cloud address, tenant, branch, and a revocable **branch key** issued from the cloud. The first connection downloads a full config snapshot, then normal sync starts. Desktop clients just point at the branch server's LAN address.

Monitor

A head-office screen lists each branch: last successful up and down sync, queue depth, version, licence expiry. It alerts when a branch has been silent too long, reusing the integration health-alert pattern.

Back up

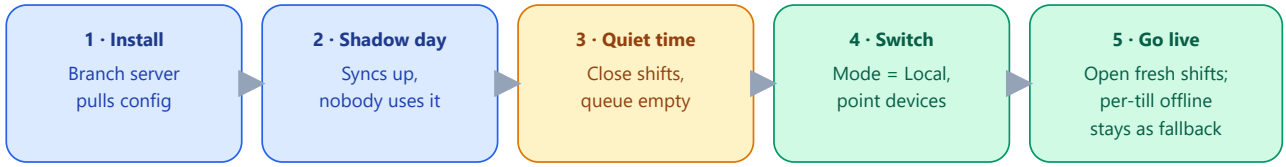
Nightly local backup (see the backup guide). The cloud copy of operational data is also an off-site backup.

Upgrade

A branch server must run a compatible version. The cloud rejects sync from a branch too far behind and tells head office. Upgrades happen branch by branch.

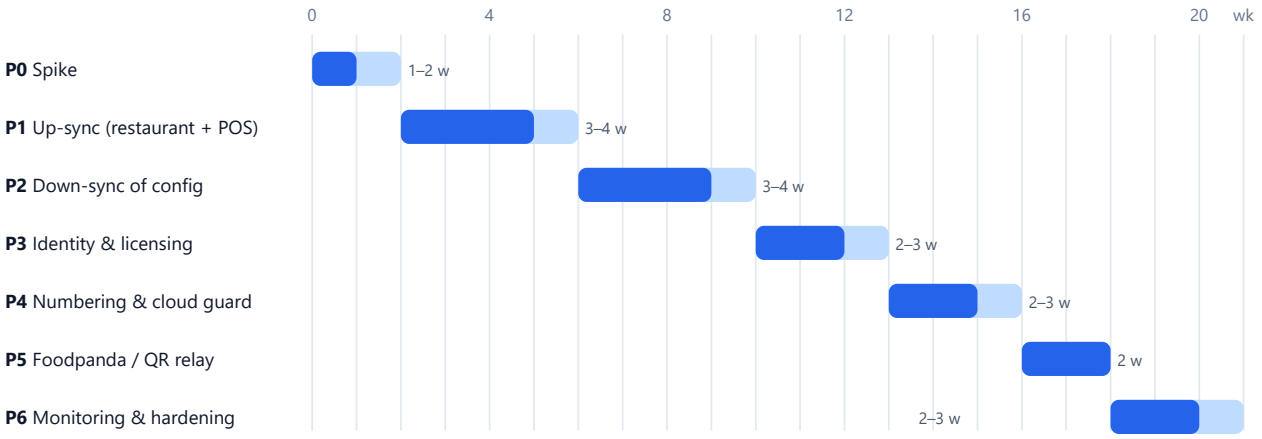
7 Moving a branch over

A deliberate, audited switch, one branch at a time. Rolling back is the reverse and is safe while the cloud copy is complete.



8 Phases and effort

Rough ranges for one developer who knows this codebase. Dark bar = minimum, light = maximum.



Run one after another by one developer. A second developer can overlap P2, P3 and P5 with P1. Total: about 15–21 weeks.

9 Decisions needed before starting

The first one decides whether this project is needed at all.

1. **Is a branch with several tills the normal case?** If every branch has a single till, today's per-till offline mode plus a 4G backup link may be enough, and this project can wait.
2. **Wallet, house account and loyalty at an offline branch:** refuse (proposed), or allow a small offline allowance per branch?
3. **Customer creation at a branch:** pending until the cloud confirms (proposed), or branch-local customers merged later?
4. **Fixing a bill after the fact:** only the branch, or head office through an "instruction" the branch applies (proposed)?
5. **Hardware:** who supplies and supports the branch PC? It needs an SSD, a UPS and a spare image.
6. **Packaging:** is the branch server a paid add-on per branch?
7. **Retail POS branches:** same phase, or restaurant first?
8. **Hosting:** confirm everything stays on erp.vrodux.com, with the branch key as the only credential a branch holds.

Not in scope

- Multi-master replication or automatic merging
- Branch-to-branch communication
- Offline Foodpanda, QR ordering or online payments
- HR, Finance, CRM and other modules at the branch
- Replacing the per-till queue — it stays as the last fallback

Risks

- **Table volume:** start with what a branch needs to trade and extend deliberately
- **Schema drift** between branch and cloud: same migrations, strict version checks
- **Support burden:** a server in every branch is operations work
- **Testing:** outages, wrong clocks, power loss mid-sync and re-seeds are the expensive part

Recommended order

1) Ship the first customer on today's per-till offline mode with a 4G link and a UPS. 2) Run the 1–2 week spike (P0) to confirm the approach. 3) Decide the packaging and the eight questions above. 4) Build P1–P6 for the first branch that genuinely needs shared offline operation.